

Pointwise provable equality and the failure of composition

Florian Lengyel*

September 30, 2026

Abstract

Montagna (1989) and Di Paola–Montagna (1991) claim that the algebraic systems S' and S'_T , respectively, are categories. We show that the proposed composition is not independent of the choice of representatives. For every consistent recursively enumerable extension T of Peano arithmetic (PA), we exhibit two program indices that are pointwise provably equal in T but yield inequivalent composites when each is run after the same program. Montagna's S' is the case $T = \text{PA}$. The failure already occurs for partial maps from ω to itself. More generally, for consistent $T \supseteq \text{PA}$, pointwise provable equality is a composition congruence exactly when T proves every true Π_1^0 sentence, in which case it is extensional equality. This completeness condition fails for every consistent recursively enumerable $T \supseteq \text{PA}$ by Gödel's second incompleteness theorem. For every extension $T \supseteq \text{PA}$, the least composition congruence containing pointwise provable equality is extensional equality if T is Σ_1^0 -sound and the universal relation otherwise.

2020 Mathematics Subject Classification.

Primary 03D75; Secondary 03F40, 08A30, 18B10.

1 Introduction

Montagna and Di Paola–Montagna propose algebraic systems S' and S'_T , asserting that S' is a pointed p -category [1, pp. 106, 108] and S'_T a p -recursion category [2, p. 646]. For every consistent recursively enumerable extension T of Peano arithmetic (PA), we show that the proposed composition is not independent of the choice of representatives. The proposed morphisms of S' and S'_T are equivalence classes of program indices under T -provable pointwise extensional equality of partial functions defined by those indices, with composition induced by composition of programs.

Montagna's S [1, p. 105] and Di Paola–Montagna's S_T [2, p. 646], which use provability of a single universally quantified equality statement, are unaffected by this obstruction.

1.1 Relation to previous work

Bergstra–Heering study ω -complete algebraic specifications, distinguishing derivability of every closed instance of an equation from derivability of the open equation [8, Sections 1.2–1.3, pp. 150–152]. The analogous distinction here is between separate proofs of equality at each standard input and a single proof of the universally quantified equality.

Program indices with a primitive recursive composition operation form a computable algebra with one binary operation. Computable quotient presentations describe an algebra by computable operations on representatives and an equivalence relation compatible with those operations; the

*Email: flengyel@gradcenter.cuny.edu

equivalence relation need not be computable [9, p. 9]. Theorem 3 determines when pointwise provable equality is a congruence for this operation, and Theorem 4 identifies the least congruence containing that equality.

Robertson–Ruškuc–Thomson prove that the monoid of unary partial recursive functions under composition is finitely generated but not finitely presented [10, Theorems 2.2–2.3, pp. 168–169]. Theorem 4 identifies this monoid as the quotient by the composition congruence generated by pointwise provable equality when T is Σ_1^0 -sound; otherwise the quotient has one element. Their results concern finite generation and finite presentation of the extensional monoid; ours identifies this monoid as the quotient by the congruence generated by pointwise provable equality when T is Σ_1^0 -sound.

2 Programs and pointwise provable equality

2.1 The constructions of Montagna and Di Paola–Montagna

Fix an acceptable Kleene numbering $(\varphi_e)_{e \in \omega}$ of the unary partial recursive functions and an arithmetization satisfying the PA-provable properties specified in Section 2.2. Throughout, a program is an index in this numbering. Thus e denotes an index, φ_e denotes the partial function computed by e , and $e(x)$ abbreviates $\varphi_e(x)$. For partial expressions, $\simeq$ means that both are undefined or both are defined with the same value. Acceptability is understood in the sense of [5, Definitions II.5.1–II.5.2, p. 215].

In the proposed construction, T is a consistent extension of PA. We take the object class and its effective coding from [1, pp. 105–106] and [2, p. 646]: the objects are generated from ω by iterated binary products and disjoint unions.

For objects A, B , the proposed morphisms $A \rightarrow B$ of S'_T are classes of program indices, called Gödel numbers in the cited papers, that compute partial recursive functions $A \multimap B$. Two indices e, d are identified, written $e \approx_{T,A} d$, precisely when, for every $a \in A$,

$$T \vdash [(\varphi_e(\bar{a}) \downarrow \leftrightarrow \varphi_d(\bar{a}) \downarrow) \wedge (\varphi_e(\bar{a}) \downarrow \rightarrow \varphi_e(\bar{a}) = \varphi_d(\bar{a}))].$$

This is the pointwise definition in [2, p. 646]. Here $\bar{a}$ denotes the numeral for the code of a ; the quantification over A is external to T . The symbols $\downarrow$ and $\uparrow$ denote convergence and divergence, respectively. Let $E_{e,d}(x)$ denote the displayed conjunction with $\bar{a}$ replaced by x ; it expresses Kleene equality at x . Montagna’s S' is the case $T = \text{PA}$; the divergence equivalence in his first conjunct gives the same relation [1, p. 106].

The proposed identity on A is the class of an index computing its identity function. Fix a primitive recursive composition-index function comp such that $\varphi_{\text{comp}(e,d)}(x) \simeq \varphi_e(\varphi_d(x))$ for all indices e, d and all x . Its existence follows from the enumeration and s - m - n theorems [5, Theorem II.1.5 and Proposition II.1.7, pp. 130–131]. For program indices f, g , write $f \bullet g := \text{comp}(f, g)$. Thus $\bullet : \omega \times \omega \rightarrow \omega$ is a fixed primitive recursive operation on indices, representing composition of partial functions: $\varphi_{f \bullet g} = \varphi_f \circ \varphi_g$. Here equality means equality of partial functions, including their domains of definition. The program with index g , the *inner program*, runs first; if it halts with output y , the program with index f , the *outer program*, runs on y .

If d and e compute partial functions $A \multimap B$ and $B \multimap D$, respectively, the intended composition is $[e] \circ [d] = [\text{comp}(e, d)]$, which must be independent of the choice of representatives. No published correction of the pointwise definitions is known to the author.

2.2 Programs and arithmetization

For $A = B = \omega$, every index of a unary partial recursive function is allowed as a representative. Write $\approx_T$ for $\approx_{T,\omega}$. For recursion-theoretic terminology, including recursive enumerability, see [7, 5]; for the arithmetical hierarchy, see [5, Definition IV.1.6, p. 367]. Truth refers to the standard model ω . A theory is Σ_1^0 -sound if every Σ_1^0 sentence it proves is true.

Alongside pointwise provable equality, we use uniform provable equality:

$$e \equiv_T d \iff T \vdash \forall x E_{e,d}(x), \quad (1)$$

$$e \approx_T d \iff \text{for every } n \in \omega, \quad T \vdash E_{e,d}(\bar{n}). \quad (2)$$

The proposed endomorphisms of ω form the quotient set $Q_T = \omega / \approx_T$; let q_T be its quotient map. Composition is independent of the choice of representatives precisely when there is a binary operation $\star$ on Q_T satisfying

$$q_T(\text{comp}(e, d)) = q_T(e) \star q_T(d) \quad \text{for all } e, d. \quad (3)$$

A composition congruence is an equivalence relation preserved by comp in both arguments. Thus a binary operation satisfying (3) exists exactly when $\approx_T$ is a composition congruence. We shall show that no such binary operation exists for consistent recursively enumerable T .

The proofs use the following arithmetization. Following Odifreddi [5, Theorem II.1.2, p. 129], write $\mathcal{T}_1(e, x, s)$ for Kleene's primitive recursive normal-form predicate for unary programs, and U for the associated primitive recursive output function. In the index convention of [5, Definition II.1.4, p. 130],

$$\varphi_e(x) \simeq U(\mu s \mathcal{T}_1(e, x, s)).$$

This is the unary case of Kleene's normal-form theorem [6, §63, Theorem XIX(a), p. 330]. The third argument s codes a halting computation of the program with index e on input x ; its output is $U(s)$. The subscript 1 indicates that the program has one input. We reserve T for the arithmetical theory. This is the Kleene normal-form convention used by Di Paola–Montagna [2, p. 645].

Define the arithmetical Σ_1^0 graph formula by

$$C_e(x, y) :\iff \exists s (\mathcal{T}_1(\bar{e}, x, s) \wedge U(s) = y).$$

Here $\mathcal{T}_1(\bar{e}, x, s)$ and $U(s) = y$ abbreviate fixed arithmetical formulas representing in PA the computation predicate and the graph of the output function, respectively. Here y denotes the output. Convergence $\varphi_e(x) \downarrow$ abbreviates $\exists y C_e(x, y)$, and divergence $\varphi_e(x) \uparrow$ its negation. Functionality gives the PA-provable equivalence

$$E_{e,d}(x) \iff \forall y (C_e(x, y) \iff C_d(x, y)).$$

By the formalized enumeration and iteration theorems in PA, invoked in [2, p. 647], the fixed comp may be taken so that its graph formula satisfies

$$\text{PA} \vdash \forall x \forall z (C_{\text{comp}(e,d)}(x, z) \iff \exists y (C_d(x, y) \wedge C_e(y, z))) \quad (4)$$

for all indices e, d . The graph formulas are provably functional in PA. More generally, if $R(x, y)$ is a Σ_1^0 formula whose functionality is provable in PA, there is an index e such that

$$\text{PA} \vdash \forall x \forall y (C_e(x, y) \iff R(x, y)). \quad (5)$$

In applications of Feferman's results, we pass to PA-provably equivalent RE-formulas in his syntactic sense [3, p. 267]. Choose such a formula $\hat{R}(z, x, y)$ with free variables exactly z, x, y and

$$\text{PA} \vdash \forall z \forall x \forall y (\hat{R}(z, x, y) \iff R(x, y)).$$

This is obtained by adjoining dummy equalities for any missing variables and taking an RE normal form that retains those free variables. Its functionality is therefore provable in Feferman's primitive recursive extension M of PA. By [3, Theorem 2.13, p. 273], there is an index e whose graph formula is provably equivalent in M to $\widehat{R}(\bar{e}, x, y)$, hence to $R(x, y)$. Eliminate the auxiliary primitive recursive definitions by [3, Theorem 2.15, p. 274] to obtain (5). Montagna uses this specialization in [1, p. 109, proof of (*)]. The graph equations below are obtained from (5).

3 An obstruction to composition

In this section, assume that $T \supseteq \text{PA}$ is consistent and recursively enumerable. Fix an index a_T for a partial recursive function whose range is the set of Gödel codes of the nonlogical axioms of T . We use Feferman's first-order calculus with equality, with the logical axiom system chosen as in [3, p. 266] so that modus ponens is the only inference rule. A derivation is a finite nonempty sequence of formulas, each a logical axiom, a nonlogical axiom of T , or obtained from earlier formulas by modus ponens; its conclusion is the last formula. At each occurrence of a nonlogical axiom with Gödel code r , a derivation code records an input m to this enumerator and a finite computation-history code s satisfying $\mathcal{T}_1(a_T, m, s)$ and $U(s) = r$. Here m is the enumerator input, s is a computation-history code, and the pair (m, s) witnesses that r is enumerated as an axiom. The relation $\text{Proof}_T(p, q)$, meaning that p codes a T -derivation whose conclusion has Gödel code q , is therefore primitive recursive. Put

$$\text{Pr}_T(q) \equiv \exists p \text{ Proof}_T(p, q).$$

For this derivation calculus, Pr_T satisfies the Hilbert–Bernays–Löb derivability conditions; see [4, pp. 115–116] for the conditions and [3, pp. 269–271, especially Theorems 2.4(i) and 2.8] for the relevant arithmetization results for RE axiom numerations. Write $\ulcorner \sigma \urcorner$ for the Gödel code of a sentence σ . Put

$$P_T(p) := \text{Proof}_T(p, \ulcorner 0 = 1 \urcorner), \quad \text{Con}(T) := \forall p \neg P_T(p).$$

Thus $P_T(p)$ says that p is a code of a T -derivation of $0 = 1$. The RE axiom numeration determined by a_T is

$$\alpha_T(r) \equiv \exists m \exists s (\mathcal{T}_1(\overline{a_T}, m, s) \wedge U(s) = r).$$

Collecting the finitely many axiom witnesses, or deleting them from an augmented derivation, gives PA-provably equivalent provability predicates for our coding and for α_T . Formalized proof manipulation, using a fixed T -proof of $0 \neq 1$, then gives $\text{PA} \vdash \text{Con}(T) \leftrightarrow \text{Con}_{\alpha_T}$, where Con_{α_T} is Feferman's consistency sentence [3, p. 269]. Let z be a program whose graph is provably empty in PA, and let id be an identity program with

$$\text{PA} \vdash \forall x \forall y (C_{\text{id}}(x, y) \longleftrightarrow x = y).$$

Theorem 1. *For every consistent recursively enumerable $T \supseteq \text{PA}$, there are programs f, g satisfying*

$$f \approx_T \text{id}, \quad f \bullet g \equiv_T z, \quad \text{id} \bullet g \equiv_T g, \quad g \not\approx_T z. \quad (6)$$

Consequently, no binary operation on Q_T satisfies (3).

Proof. Choose

$$f(n) = \begin{cases} n, & \neg P_T(n), \\ \uparrow, & P_T(n), \end{cases} \quad g(x) = \mu p P_T(p).$$

Choose their indices so that PA proves

$$C_f(n, y) \longleftrightarrow \neg P_T(n) \wedge y = n, \quad (7)$$

$$C_g(x, p) \longleftrightarrow P_T(p) \wedge \forall r < p \neg P_T(r). \quad (8)$$

Fix $n \in \omega$. Consistency of T implies that $P_T(n)$ is false. Since P_T is primitive recursive, PA verifies this closed instance: $\text{PA} \vdash \neg P_T(\bar{n})$. Hence $\text{PA} \vdash f(\bar{n}) = \bar{n}$. Thus $f \approx_T \text{id}$.

By (4), (7), and (8), convergence of $f \bullet g$ would require an intermediate value p satisfying both $P_T(p)$ and $\neg P_T(p)$. Hence

$$\text{PA} \vdash \forall x (f \bullet g)(x) \uparrow, \quad (9)$$

so $f \bullet g \equiv_T z$. The identity law $\text{id} \bullet g \equiv_T g$ is also provable in PA.

By (8),

$$\text{PA} \vdash (g(0) \uparrow \longleftrightarrow \text{Con}(T)).$$

If $g \approx_T z$, the instance at input 0 would therefore yield $T \vdash \text{Con}(T)$, contrary to Gödel's second incompleteness theorem [3, Theorem 2.7(ii), p. 271]. This proves all four assertions in (6).

Finally, if $\star$ satisfied (3), then

$$\begin{aligned} q_T(g) &= q_T(\text{id} \bullet g) = q_T(\text{id}) \star q_T(g) \\ &= q_T(f) \star q_T(g) = q_T(f \bullet g) = q_T(z), \end{aligned}$$

contradicting $g \not\approx_T z$. □

Under the consistency assumption, $\varphi_f = \varphi_{\text{id}}$ and $\varphi_g = \varphi_z$: f computes the identity and g is nowhere defined. Nevertheless, $g \not\approx_T z$. The obstruction concerns composition on pointwise provability classes; composition of the underlying partial functions remains well defined.

Corollary 2. *For consistent recursively enumerable $T \supseteq \text{PA}$, the algebraic system S'_T is not a category under the proposed composition. The failure already occurs for indices representing partial maps $\omega \rightarrow \omega$. In particular, it occurs for Montagna's S' with $T = \text{PA}$.*

The categorical associativity and identity axioms are not needed for this contradiction. Any composition congruence containing $\approx_T$ must identify g with z , and therefore strictly enlarge $\approx_T$. Theorem 4 determines the least such congruence.

The assertion that S'_T is locally connected if and only if T is Σ_1^0 -sound [2, Theorem 4.5, pp. 658–659] presupposes that S'_T is a category.

3.1 A proof from productivity

Di Paola–Heller prove a creativity theorem for the diagonal halting domain in dominical recursion categories [11, Theorem 8.5, pp. 622–623].

The existence assertion in Theorem 1 also follows from the productivity of the diagonal nonhalting set [12, Section 5.3, p. 136]. Put

$$K = \{e : \varphi_e(e) \downarrow\}, \quad D_T = \{e : T \vdash \varphi_e(\bar{e}) \uparrow\}, \quad W_e = \text{dom}(\varphi_e).$$

The set D_T is recursively enumerable. Consistency gives $D_T \subseteq \overline{K}$, since every actual halting computation is verifiable in PA. Choose d with $W_d = D_T$. If $d \in D_T$, then $\varphi_d(d) \downarrow$, contradicting $D_T \subseteq \overline{K}$. Hence

$$d \in \overline{K} \setminus D_T.$$

This is the productive-set argument: whenever $W_e \subseteq \overline{K}$, the index e itself belongs to $\overline{K} \setminus W_e$.

Let $R(n) \equiv \neg \mathcal{T}_1(\bar{d}, \bar{d}, n)$, and choose programs

$$f(n) = \begin{cases} n, & R(n), \\ \uparrow, & \neg R(n), \end{cases} \quad g(x) = \mu n \neg R(n),$$

with PA-provable graph equations

$$C_f(n, y) \longleftrightarrow R(n) \wedge y = n, \quad C_g(x, n) \longleftrightarrow \neg R(n) \wedge \forall k < n R(k).$$

Every standard instance $R(\bar{n})$ is PA-provable, so $f \approx_T \text{id}$. The graph equations give $f \bullet g \equiv_T z$ and $\text{id} \bullet g \equiv_T g$. By the definition of C_d and the PA-provable totality of U ,

$$\text{PA} \vdash (\varphi_d(\bar{d}) \uparrow \longleftrightarrow \forall n R(n)).$$

Together with the graph equation for g and the definition of D_T , this gives

$$g \approx_T z \iff T \vdash \forall n R(n) \iff d \in D_T,$$

which is false. Thus all four assertions of (6) hold. The proof using second incompleteness selects the particular unprovable sentence $\text{Con}(T)$; productivity supplies another true unprovable divergence sentence.

4 When pointwise provable equality is a congruence

For any $T \supseteq \text{PA}$, left composition with a fixed program f is compatible with $\approx_T$: replacing the inner program gives $g \approx_T h \Rightarrow f \bullet g \approx_T f \bullet h$. Indeed, for each $n \in \omega$, substitute the theorem $T \vdash \forall y (C_g(\bar{n}, y) \leftrightarrow C_h(\bar{n}, y))$ into (4). Right composition with a fixed program g instead replaces the outer program: $f \approx_T h$ would have to imply $f \bullet g \approx_T h \bullet g$. Theorem 1 gives $f \approx_T \text{id}$ but $f \bullet g \not\approx_T \text{id} \bullet g$. The family $T \vdash E_{f,h}(\bar{m})$ ($m \in \omega$) need not yield a single T -derivation of $\forall y E_{f,h}(y)$. Such a derivation would justify replacing $C_f(y, z)$ by $C_h(y, z)$ beneath the existential quantifier over the intermediate output in (4).

Theorem 3. *For any consistent $T \supseteq \text{PA}$, not necessarily recursively enumerable, the following are equivalent.*

1. *Pointwise provable equality is preserved by right composition: $f \approx_T h$ implies $f \bullet g \approx_T h \bullet g$ for every program index g .*
2. *The relation $\approx_T$ is a composition congruence.*
3. *The theory T proves every true Π_1^0 sentence.*
4. *For all e, d , $e \approx_T d$ if and only if φ_e and φ_d are extensionally equal as partial functions on ω .*

Proof. We first prove $1 \Rightarrow 3$. Let θ be a true Π_1^0 sentence. Choose a primitive recursive predicate R with a decision procedure whose correctness is provable in PA, such that

$$\text{PA} \vdash \theta \longleftrightarrow \forall m R(m).$$

Define programs

$$f(n) = \begin{cases} n, & R(n), \\ \uparrow, & \neg R(n), \end{cases} \quad g(x) = \mu m \neg R(m).$$

Choose their indices so that PA proves

$$C_f(n, y) \longleftrightarrow R(n) \wedge y = n, \quad (10)$$

$$C_g(x, m) \longleftrightarrow \neg R(m) \wedge \forall k < m R(k). \quad (11)$$

For each $n \in \omega$, the PA-verified decision procedure for R gives $\text{PA} \vdash R(\bar{n})$. Hence $\text{PA} \vdash f(\bar{n}) = \bar{n}$, and $f \approx_T \text{id}$.

Equations (4), (10), and (11) show that PA proves that $f \bullet g$ is everywhere undefined: any intermediate output m would have to satisfy both $\neg R(m)$ and $R(m)$. Thus $f \bullet g \equiv_T z$. By (11),

$$\text{PA} \vdash \forall x (g(x) \uparrow \longleftrightarrow \theta).$$

Consequently,

$$g \approx_T z \iff T \vdash \theta.$$

The forward implication uses the instance at input 0; the reverse implication follows from the displayed equivalence. Assumption 1, applied to $f \approx_T \text{id}$, now gives $f \bullet g \approx_T g$ by the PA-provable identity law. Hence $g \approx_T z$ and $T \vdash \theta$.

Next assume 3, and fix indices e, d such that φ_e and φ_d are extensionally equal. Fix $n \in \omega$. If $\varphi_e(n) = \varphi_d(n) = y$, computation-history codes yield PA-proofs of $C_e(\bar{n}, \bar{y})$ and $C_d(\bar{n}, \bar{y})$; PA-provable functionality then yields $\text{PA} \vdash E_{e,d}(\bar{n})$. If both functions are undefined at n , the two divergence assertions are true Π_1^0 sentences, so T proves them and hence proves $E_{e,d}(\bar{n})$. Thus extensional equality implies $\approx_T$.

Conversely, suppose that φ_e and φ_d are extensionally unequal. Choose $n \in \omega$ at which they differ. If both are defined there with distinct outputs, their computation-history codes and PA-provable functionality yield a PA-proof of $\neg E_{e,d}(\bar{n})$. If exactly one is defined there, its computation-history code yields a PA-proof of the corresponding graph instance, while the other program's divergence at n is a true Π_1^0 sentence and hence is provable in T . Thus $T \vdash \neg E_{e,d}(\bar{n})$ in either case. Consistency excludes $T \vdash E_{e,d}(\bar{n})$. This proves 4.

Finally, extensional equality is a composition congruence, so $4 \Rightarrow 2$, and $2 \Rightarrow 1$ is immediate. $\square$

For consistent recursively enumerable $T \supseteq \text{PA}$, $\text{Con}(T)$ is a true Π_1^0 sentence unprovable in T by Gödel's second incompleteness theorem. Thus condition 3 fails, and $\approx_T$ is not a composition congruence. Taking $\theta = \text{Con}(T)$ in the first implication recovers the programs of Theorem 1. The theory obtained by adjoining all true Π_1^0 sentences to PA satisfies condition 3; it is consistent and is not recursively enumerable.

5 The generated composition congruence

For an extension $T \supseteq \text{PA}$, let $\sim_T$ be the least composition congruence on program indices containing $\approx_T$; it is the intersection of all such congruences.

Theorem 4 (Generated composition congruence). *Let $T \supseteq \text{PA}$, not necessarily consistent or recursively enumerable. Then*

$$\sim_T = \begin{cases} \{(e, d) \in \omega^2 : \varphi_e = \varphi_d\}, & \text{if } T \text{ is } \Sigma_1^0\text{-sound,} \\ \omega \times \omega, & \text{otherwise.} \end{cases}$$

Proof. First we prove that extensional equality is contained in $\sim_T$. Fix e, d with $\varphi_e = \varphi_d$. Choose a primitive recursive bijective pairing $\langle s, x \rangle$ with primitive recursive projections whose inverse identities are provable in PA. Put

$$D_e(x, s) \equiv \mathcal{T}_1(\bar{e}, x, s) \wedge \forall r < s \neg \mathcal{T}_1(\bar{e}, x, r).$$

Choose program indices $H_e, B_e, A_{e,d}$ so that PA proves the universal closures of

$$\begin{aligned} C_{H_e}(x, n) &\longleftrightarrow \exists s (D_e(x, s) \wedge n = \langle s, x \rangle), \\ C_{B_e}(\langle s, x \rangle, y) &\longleftrightarrow \mathcal{T}_1(\bar{e}, x, s) \wedge U(s) = y, \\ C_{A_{e,d}}(\langle s, x \rangle, y) &\longleftrightarrow \mathcal{T}_1(\bar{e}, x, s) \wedge U(s) = y \wedge C_d(x, y). \end{aligned}$$

These are Σ_1^0 relations whose functionality is provable in PA, so (5) supplies the required indices. The program H_e returns the pair consisting of the least computation-history code for e on its input and the input itself.

For each standard pair $\langle s, x \rangle$, PA proves the true instance of either $\mathcal{T}_1(e, x, s)$ or its negation. If $\mathcal{T}_1(e, x, s)$ is false, the graph equations prove that both $A_{e,d}$ and B_e are undefined there. If it is true, then $\varphi_d(x) = \varphi_e(x) = U(s)$. A computation-history code for d on input x therefore gives a PA-proof of $C_d(\bar{x}, U(s))$. The graph equations and functionality give PA-provable equality of the two programs at that pair. Thus $A_{e,d} \approx_{\text{PA}} B_e$.

The least-number principle and functionality of C_e give

$$\text{PA} \vdash \forall x \forall y (C_e(x, y) \longleftrightarrow \exists s (D_e(x, s) \wedge U(s) = y)).$$

Indeed, if $C_e(x, y)$ holds, the least s satisfying $\mathcal{T}_1(\bar{e}, x, s)$ also satisfies $C_e(x, U(s))$, so functionality forces $U(s) = y$. Equation (4) now gives $B_e \bullet H_e \equiv_{\text{PA}} e$ and

$$\text{PA} \vdash \forall x \forall y (C_{A_{e,d} \bullet H_e}(x, y) \longleftrightarrow C_e(x, y) \wedge C_d(x, y)).$$

Since $\equiv_{\text{PA}} \subseteq \approx_T \subseteq \sim_T$, composition compatibility gives $e \sim_T A_{e,d} \bullet H_e$. Repeating the construction with e, d interchanged gives $d \sim_T A_{d,e} \bullet H_d$. The two composite graph formulas are provably equivalent in PA, by symmetry of conjunction. Hence $e \sim_T d$.

Suppose next that T is Σ_1^0 -sound. If $e \approx_T d$ and $\varphi_e(n) = m$, a computation-history code gives $\text{PA} \vdash C_e(\bar{n}, \bar{m})$. Together with $T \vdash E_{e,d}(\bar{n})$ this yields $T \vdash C_d(\bar{n}, \bar{m})$. The latter sentence is Σ_1^0 , so it is true and $\varphi_d(n) = m$. Interchanging e, d shows that $\approx_T$ is contained in extensional equality. Extensional equality is a composition congruence, so minimality of $\sim_T$, together with the first part, proves the first case.

Finally, suppose that T proves a false Σ_1^0 sentence σ . Choose a program δ such that

$$\text{PA} \vdash \forall x \forall y (C_\delta(x, y) \longleftrightarrow \sigma \wedge y = x).$$

This graph is Σ_1^0 and provably functional in PA. Since $T \vdash \sigma$, we have $\delta \equiv_T \text{id}$. Since σ is false, $\varphi_\delta = \varphi_z$, so the first part gives $\delta \sim_T z$. Thus $\text{id} \sim_T z$, and for every program index e ,

$$e \equiv_{\text{PA}} e \bullet \text{id} \sim_T e \bullet z \equiv_{\text{PA}} z.$$

Therefore $\sim_T$ is the universal relation. □

The quotient $\omega / \sim_T$ is therefore the monoid of unary partial recursive functions when T is Σ_1^0 -sound, and the one-element monoid otherwise. In particular, closing $\approx_{\text{PA}}$ under composition gives exactly extensional equality.

6 Weak totality and ranges

For consistent recursively enumerable $T \supseteq \text{PA}$, weak totality and the range assignment [1, Definition 1.1, p. 106, and p. 111] are not invariant under $\approx_T$.

6.1 Weak totality

For a program index e , define the predicate

$$W_T(e) \iff \text{for every program index } d, \quad \text{comp}(e, d) \approx_T z \implies d \approx_T z. \quad (12)$$

This is the index formulation of weak totality in Di Paola–Montagna [2, Definition 1.5, p. 645]. Montagna calls a morphism satisfying this condition DPH total [1, Definition 1.1, p. 106].

Let d_e be a program index for the partial identity on the domain of φ_e , with PA-provable graph equation

$$C_{d_e}(x, y) \longleftrightarrow x = y \wedge \exists v C_e(x, v).$$

The condition $d_e \approx_T \text{id}$ is the index condition corresponding to R totality in Montagna’s terminology [1, Definitions 1.6–1.7, p. 107] and to totality in the terminology of Di Paola–Montagna [2, Definition 1.4, p. 645].

Proposition 5. *For consistent recursively enumerable $T \supseteq \text{PA}$, the predicate W_T is not invariant under $\approx_T$.*

Proof. The PA-provable identity law gives $W_T(\text{id})$. For f, g in Theorem 1, $f \bullet g \approx_T z$ but $g \not\approx_T z$, so $W_T(f)$ fails, although $f \approx_T \text{id}$. $\square$

For $T = \text{PA}$, the graph equation for f has the form used for the first program in Montagna’s Example 2 [1, p. 112], with the chosen predicate $P_{\text{PA}}(x)$ in place of Montagna’s $\text{Prf}_{\text{PA}}(x, \ulcorner 0 = 1 \urcorner)$. Montagna defines Prf_{PA} relative to a chosen binumeration of the axioms of PA [1, note 2, p. 115]. Montagna states that the class of his first program is R total and DPH total in S' . The corresponding program f above satisfies the R-totality condition $d_f \approx_{\text{PA}} \text{id}$ but fails W_{PA} .

The second program displayed in Montagna’s example is the partial identity with domain

$$\{p : \text{Prf}_{\text{PA}}(p, \ulcorner 0 = 1 \urcorner)\},$$

the codes, in Montagna’s proof coding, of PA-derivations of $0 = 1$. It is pointwise provably equal to z . By contrast, the unbounded-search program g defined above satisfies $f \bullet g \approx_{\text{PA}} z$ and $g \not\approx_{\text{PA}} z$.

An arithmetical sentence σ is Π_1^0 -conservative over PA if every Π_1^0 sentence provable in $\text{PA} + \sigma$ is already provable in PA. In [1, Theorem 2.3, p. 111], condition (2) says that the convergence sentence $f(\bar{n}) \downarrow$ is Π_1^0 -conservative over PA for every $n \in \omega$. Each such sentence is already PA-provable, so this condition holds. Condition (1), read as (12), fails. Thus the implication (2) $\Rightarrow$ (1) fails when interpreted in terms of program indices.

6.2 The range assignment

For each program index e , choose a program index ρ_e for the partial identity whose domain is the set-theoretic range of φ_e . With r as input and w as output, its PA-provable graph equation is

$$C_{\rho_e}(r, w) \longleftrightarrow r = w \wedge \exists x C_e(x, r). \quad (13)$$

This is the range construction defined in [1, p. 111]; the range assertion for S'_T is repeated in [2, p. 646].

Proposition 6. *For consistent recursively enumerable $T \supseteq \text{PA}$, there is a program u with $u \approx_T z$ but $\rho_u \not\approx_T \rho_z$. Thus (13) does not define an operation on Q_T .*

Proof. Let

$$u(p) = \begin{cases} 0, & P_T(p), \\ \uparrow, & \neg P_T(p) \end{cases},$$

with PA-provable graph equation

$$\forall p \forall y (C_u(p, y) \longleftrightarrow P_T(p) \wedge y = 0).$$

For every $p \in \omega$, PA proves $\neg P_T(\bar{p})$ and $u(\bar{p}) \uparrow$. Hence $u \approx_T z$. But (13) yields

$$\text{PA} \vdash \forall r \forall w (C_{\rho_u}(r, w) \longleftrightarrow r = w \wedge r = 0 \wedge \exists p P_T(p)).$$

In particular, $\text{PA} \vdash C_{\rho_u}(0, 0) \leftrightarrow \exists p P_T(p)$. The program ρ_z is PA-provably everywhere undefined. If $\rho_u \approx_T \rho_z$, the instance at input 0 gives $T \vdash \neg \exists p P_T(p)$, again contradicting the second incompleteness theorem. $\square$

Use of AI tools. The author used ChatGPT¹ and Claude to explore mathematical arguments and related literature and to revise drafts. The author takes responsibility for the mathematical arguments and references.

References

- [1] F. Montagna, “Pathologies” in two syntactic categories of partial maps, *Notre Dame Journal of Formal Logic* **30** (1989), no. 1, 105–116. [doi:10.1305/ndjfl/1093634998](https://doi.org/10.1305/ndjfl/1093634998).
- [2] R. A. Di Paola and F. Montagna, *Some properties of the syntactic p-recursion categories generated by consistent, recursively enumerable extensions of Peano arithmetic*, *Journal of Symbolic Logic* **56** (1991), no. 2, 643–660. [doi:10.2307/2274707](https://doi.org/10.2307/2274707).
- [3] S. Feferman, *Transfinite recursive progressions of axiomatic theories*, *Journal of Symbolic Logic* **27** (1962), no. 3, 259–316. [doi:10.2307/2964649](https://doi.org/10.2307/2964649).
- [4] M. H. Löb, *Solution of a problem of Leon Henkin*, *Journal of Symbolic Logic* **20** (1955), no. 2, 115–118. [doi:10.2307/2266895](https://doi.org/10.2307/2266895).
- [5] P. Odifreddi, *Classical recursion theory: The theory of functions and sets of natural numbers*, North-Holland Publishing Co., Amsterdam, 1989.
- [6] S. C. Kleene, *Introduction to Metamathematics*, Bibliotheca Mathematica, vol. 1, North-Holland, Amsterdam, 1952.
- [7] H. Rogers, Jr., *Theory of recursive functions and effective computability*, McGraw-Hill Book Co., New York, 1967.
- [8] J. A. Bergstra and J. Heering, *Which data types have ω -complete initial algebra specifications?*, *Theoretical Computer Science* **124** (1994), 149–168. [doi:10.1016/0304-3975\(94\)90057-4](https://doi.org/10.1016/0304-3975(94)90057-4).
- [9] M. T. Godziszewski and J. D. Hamkins, *Computable quotient presentations of models of arithmetic and set theory*, *RIMS Kôkyûroku* **2050** (2017), 9–23. <https://www.kurims.kyoto-u.ac.jp/~kyodo/kokyuroku/contents/pdf/2050-02.pdf>.

¹Some draft reviews used the HostileRefereeGPT instructions: <https://github.com/flengyel/HostileRefereeGPT>.

- [10] E. F. Robertson, N. Ruškuc, and M. R. Thomson, *On diagonal acts of monoids*, Bulletin of the Australian Mathematical Society **63** (2001), no. 1, 167–175. doi:10.1017/S0004972700019225.
- [11] R. A. Di Paola and A. Heller, *Dominical categories: recursion theory without elements*, Journal of Symbolic Logic **52** (1987), no. 3, 594–635. doi:10.2307/2274352.
- [12] J. Gallier and J. Quaintance, *Introduction to the theory of computation: Computability, complexity, and the lambda calculus*, lecture notes, University of Pennsylvania, April 28, 2020. <https://www.cis.upenn.edu/~cis2620/notes/tcbook-u-comp.pdf>.